

Student name: Jason Mark Vellucci

Student ID: 221437402

SIT225: Data Capture Technologies

Activity 5.1: Firebase Realtime database

The Firebase Realtime Database is a cloud-hosted NoSQL database that lets you store and sync data between your users in real-time. Data is stored as JSON and synchronized in real-time to every connected client. In this activity, you will set up and perform operations such as queries and updates on the database using Python programming language.

Hardware Required

No hardware is required.

Software Required

Firebase Realtime database
Python 3

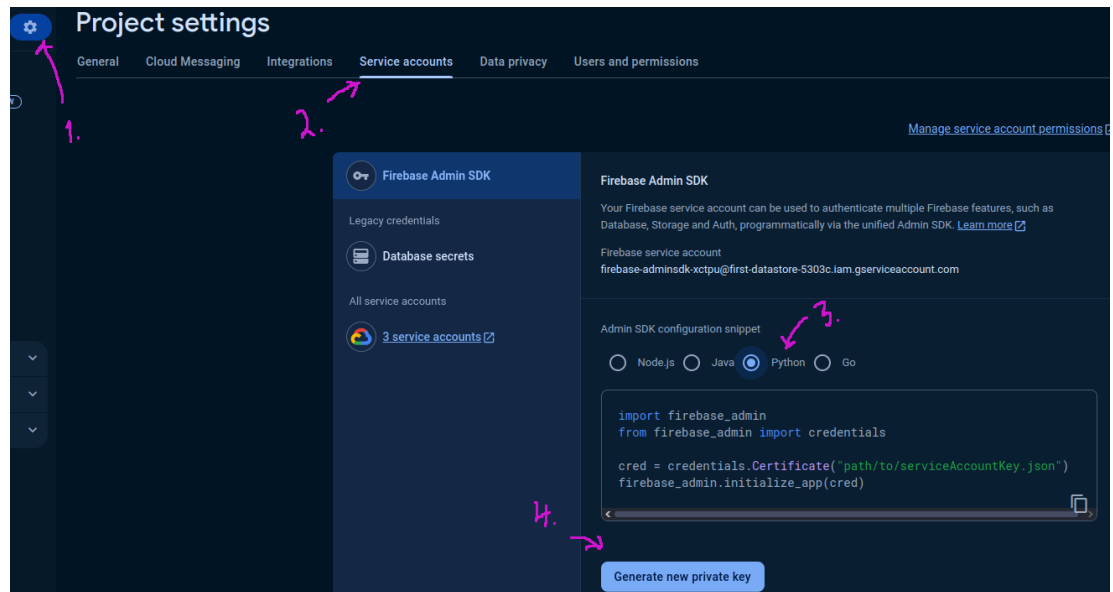
Steps

Step	Action
1	Create an Account: First, you will need to create an account in the Firebase console, follow instructions in the official Firebase document (https://firebase.google.com/docs/database/rest/start).
2	Create a Database: Follow the above Firebase document to create a database. When you click on Create Database, you have to specify the location of the database and the security rules. Two rules are available – locked mode and test mode; since we will be using the database for reading, writing, and editing, we choose test mode.
3	Setup Python library for Firebase access: We will be using Admin Database API, which is available in <i>firebase_admin</i> library. Use the below command in the command line to install. You can follow a Firebase tutorial here

(<https://www.freecodecamp.org/news/how-to-get-started-with-firebase-using-python>).

```
$ pip install firebase_admin
```

Firebase will allow access to Firebase server APIs from Google Service Accounts. To authenticate the Service Account, we require a private key in JSON format. To generate the key, go to project settings, click Generate new private key, download the file, and place it in your current folder where you will create your Python script.



4

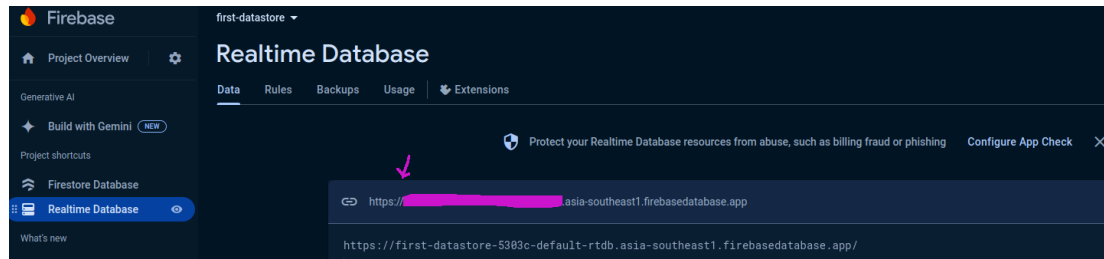
Connect to Firebase using Python version of Admin Database API:

A credential object needs to be created to initialise the Python library which can be done using the Python code below. Python notebook can be downloaded here

(https://github.com/deakin-deep-dreamer/sit225/blob/main/week_5/firebase_explore.ipynb).

```
1 import firebase_admin
2
3 databaseURL = 'https://XXX.firebaseio.com/'
4 cred_obj = firebase_admin.credentials.Certificate(
5     'first-datastore-5303c-firebase-adminsdk-xctpu-c9902044ac.json'
6 )
7 default_app = firebase_admin.initialize_app(cred_obj, {
8     'databaseURL': databaseURL
9 })
```

The databaseURL is a web address to reach your Firebase database that you have created in step 2. This URL can be found in the Data tab of Realtime Database.

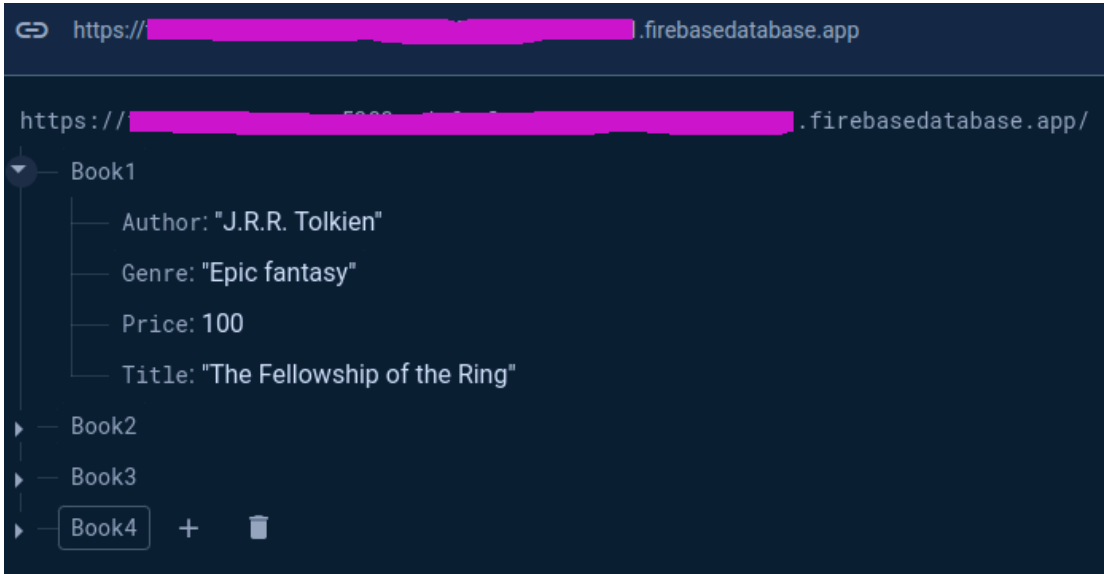


If you compile the code snippet above, it should do with no error.

5 Write to database Using the set() Function:

We set the reference to the root of the database (or we could also set it to a key value or child key value). Data needs to be in JSON format as below.

```
1 from firebase_admin import db
2
3 # A reference point is always needed to be set
4 # before any operation is carried out on a database.
5 #
6 ref = db.reference("/")
7
8 # JSON format data (key/value pair)
9 data = { # Outer {} contains inner data structure
10     "Book1":
11         {
12             "Title": "The Fellowship of the Ring",
13             "Author": "J.R.R. Tolkien",
14             "Genre": "Epic fantasy",
15             "Price": 100
16         },
17     "Book2":
18         {
19             "Title": "The Two Towers",
20             "Author": "J.R.R. Tolkien",
21             "Genre": "Epic fantasy",
22             "Price": 100
23         },
24     "Book3":
25         {
26             "Title": "The Return of the King",
27             "Author": "J.R.R. Tolkien",
28             "Genre": "Epic fantasy",
29             "Price": 100
30         },
31     "Book4":
32         {
33             "Title": "Brida",
34             "Author": "Paulo Coelho",
35             "Genre": "Fiction",
36             "Price": 100
37         }
38 }
39
40 # JSON format data is set (overwritten) to the reference
41 # point set at /, which is the root node.
42 #
43 ref.set(data)
```

	A reference point always needed to be set where the data read/write will take place. In the code above, the reference point is set at the root of the NoSQL Document, where consider the database is a JSON tree and / is the root node of the tree). The set() function writes (overwrites) data at the set reference point. You can visualise the data in the Firebase console as below -  <pre>https://[redacted].firebase.database.app/ { "Book1": { "Author": "J.R.R. Tolkien", "Genre": "Epic fantasy", "Price": 100, "Title": "The Fellowship of the Ring" }, "Book2": {}, "Book3": {}, "Book4": {} }</pre>
6	Read data using get() function: Data can be read using get() function on the reference set beforehand, as shown below.

	<pre> 1 ref = db.reference("/") # set ref point 2 3 # query all data under the ref 4 books = ref.get() 5 print(books) 6 print(type(books)) 7 8 # print each item separately 9 for key, value in books.items(): 10 print(f"{key}: {value}") 11 12 13 # Query /Book1 14 ref = db.reference("/Book1") 15 books = ref.get() 16 print(books) </pre> ✓ 0.3s <pre> {'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'}} <class 'dict'> Book1: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} Book2: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} Book3: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} Book4: {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'} {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Hobbit'} </pre> Consider the reference set in line 1 and the output compared to the reference set at line 14 and the bottom output line to understand the use of <code>db.reference()</code> and <code>ref.get()</code>.
7	Write to database Using the push() Function: The <code>push()</code> function saves data under a <i>unique system generated key</i>. This is different than <code>set()</code> where you set the keys such as Book1, Book2, Book3 and Book4 under which the content (author, genre, price and title) appears. Let's try to push the same data in the root reference. Note that since we already has data under root / symbol, setting (or pushing) in the same reference point will eventually rewrite the original data.

```

1 # Write using push() function
2 # Note that a set() is called on top of push()
3 #
4 ref = db.reference("/")
5 ref.set({
6     "Books":
7     {
8         "Best_Sellers": -1
9     }
10 })
11
12 ref = db.reference("/Books/Best_Sellers")
13
14 for key, value in data.items():
15     ref.push().set(value)

```

✓ 2.0s

The output will reset the previous data set in / node. The current data is shown below.



As you can see, under /Books/Best_Sellers there are 4 nodes where the node head (or node ID) is a randomly generated key which is due to the use of push() function. When data key does not matter, the use of push() function is desirable.

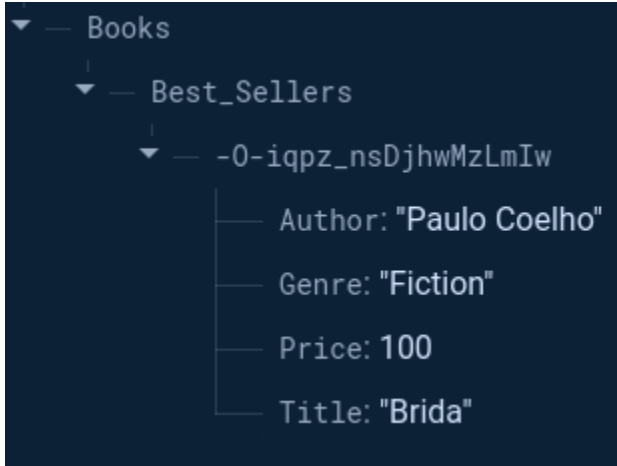
Let's say the price of the books by J. R. R. Tolkien is reduced to 80 units to offer a discount. The first 3 books are written by this author, and we want to apply for a discount on all of them.

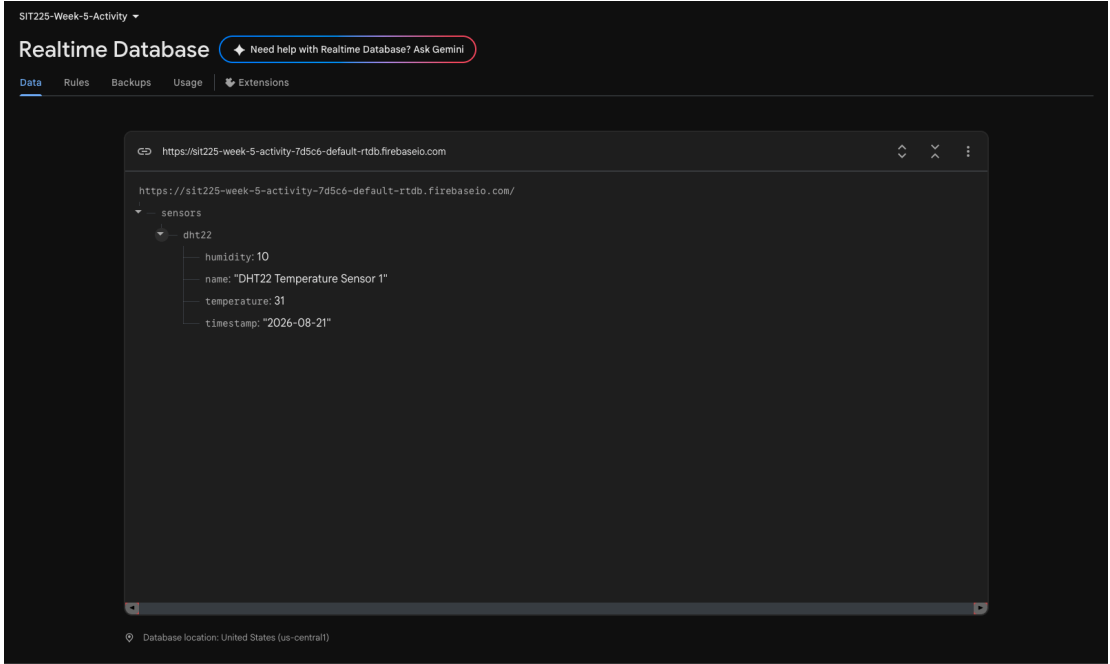
```
1  # Update data
2  #
3  # Requirement: The price of the books by
4  # J. R. R. Tolkien is reduced to 80 units to
5  # offer a discount.
6  #
7  ref = db.reference("/Books/Best_Sellers/")
8  best_sellers = ref.get()
9  print(best_sellers)
10 for key, value in best_sellers.items():
11     if(value["Author"] == "J.R.R. Tolkien"):
12         value["Price"] = 90
13         ref.child(key).update({"Price":80})
```

✓ 0.9s

As you can see, the author name is compared and the new price is set in the best_sellers dictionary and finally, an update() function is called on the ref, however, the current ref is a '/Books/Best_Sellers/', so we need to locate the child under the ref node, so ref.child(key) is used in line 13. The output is shown below with a discounted price.

	
9	Delete data: Let's delete all bestseller books with J.R.R. Tolkien as the author. You can locate the node using <code>db.reference()</code> (line 4) and then locate specific record (for loop in line 6) and calling <code>set()</code> with empty data <code>{}</code> as a parameter, such as <code>set({})</code>. The particular child under the ref needs to be located first by using <code>ref.child(key)</code>, otherwise, the ref node will be removed – BE CAREFUL. <pre data-bbox="509 1596 1227 1885"> 1 # Let's delete all best seller books 2 # with J.R.R. Tolkien as the author. 3 # 4 ref = db.reference("/Books/Best_Sellers") 5 6 for key, value in best_sellers.items(): 7 if(value["Author"] == "J.R.R. Tolkien"): 8 ref.child(key).set({}) </pre>

	This keeps only the other author data, as shown below.  If ref.child() not used, as shown the code below, all data will be removed. <pre> 1 ref = db.reference("/Books/Best_Sellers") 2 ref.set({}) </pre> Now in Firebase console you will see no data exists.
10	Question: Run all the cells in the Notebook you have downloaded in Step 4, fill in the student information at the top cell of the Notebook. Convert the Notebook to PDF and merge with this activity sheet PDF. Answer: Convert the Notebook to PDF and merge with this activity sheet PDF.
10	Question: Create a sensor data structure for DHT22 sensor which contains attributes such as sensor_name, timestamp, temperature and humidity. Remember there will be other sensors with different sensor variables such as DHT22 has 2 variables, accelerometer sensor has 3. For each such sensor, you will need to gather data over time. Discuss how you are going to handle multiple data values in JSON format? Justify your design. Answer: <pre> { "sensors": { "dht22": { "name", "timestamp", "temperature", "humidity" }, </pre>

	<pre>...moreSensorsHere, } }</pre> The top-level `sensors` object encapsulates the child-sensor data. This is extensible because for each new sensor, we can just set the new sensor on the `sensors` key and set whichever keys we need for the new sensor on the new sensor object.
11	Question: Generate some random data for DHT22 sensor, insert data to database, query all data and screenshot the output here. Answer:  The screenshot shows the Firebase Realtime Database interface. The URL bar displays the database path: <code>https://sit225-week-5-activity-7d5c6-default-rtdb.firebaseio.com</code>. The left sidebar shows the database structure with a 'sensors' node expanded, revealing a 'dht22' node. The 'dht22' node contains the following JSON data: <code>{humidity: 10, name: "DHT22 Temperature Sensor 1", temperature: 31, timestamp: "2026-08-21"}</code>. The bottom status bar indicates the database location is 'United States (us-central1)'.
12	Question: Generate some random data for the SR04 Ultrasonic sensor, insert data to database, query all data and screenshot the output here. Answer:  The screenshot shows a code editor with a JavaScript snippet for querying a Firebase database. The code is as follows: <pre>[10]: ref = db.reference("/sensors/sr04"); sr04 = ref.get(); print(sr04)</pre> Below the code, the output is displayed as a JSON object: <code>{'distance_cm': 42.7, 'name': 'HC-SR04 Ultrasonic Sensor 1', 'timestamp': '2026-08-21'}</code>.
13	Question: Firebase Realtime database generates events on data operations. You can refer to section 'Handling Realtime Database events' in the document (https://firebase.google.com/docs/functions/database-events?gen=2nd).

Discuss in the active learning session and summarise the idea of database events and how it is handled using Python SDK.

Note that these events are useful when your sensors (from Arduino script) store data directly to Firebase Realtime database and you would like to track data update actions from a central Python application such as a monitoring dashboard.

Answer: This is essentially EDA (Event Driven Architecture). The idea is to `listen` to the firing of `events` (in this context, certain database operations), and to `handle` those events using `Cloud Functions`.

In the context of sensors, this could mean that when a certain sensor object's key is updated in the Realtime Database, an event is fired and the Cloud Function triggers. That cloud function's implementation may, for example, propagate that changed value to an Arduino Cloud dashboard.

For Python specifically, we can use the Firebase SDKs Cloud Function library. Certain Db events correspond to a predefined interface, such as `on\_value\_updated()`. We define that function, deploy it to the cloud using the Firebase CLI, and whenever a Db update occurs the deployed handler function will run.

Activity 5.2: Data wrangling

Data wrangling is the process of converting raw data into a usable form. The process includes collecting, processing, analyzing, and tidying the raw data so that it can be easily read and analyzed. In this activity, you will use the common library in python, "pandas".

Hardware Required

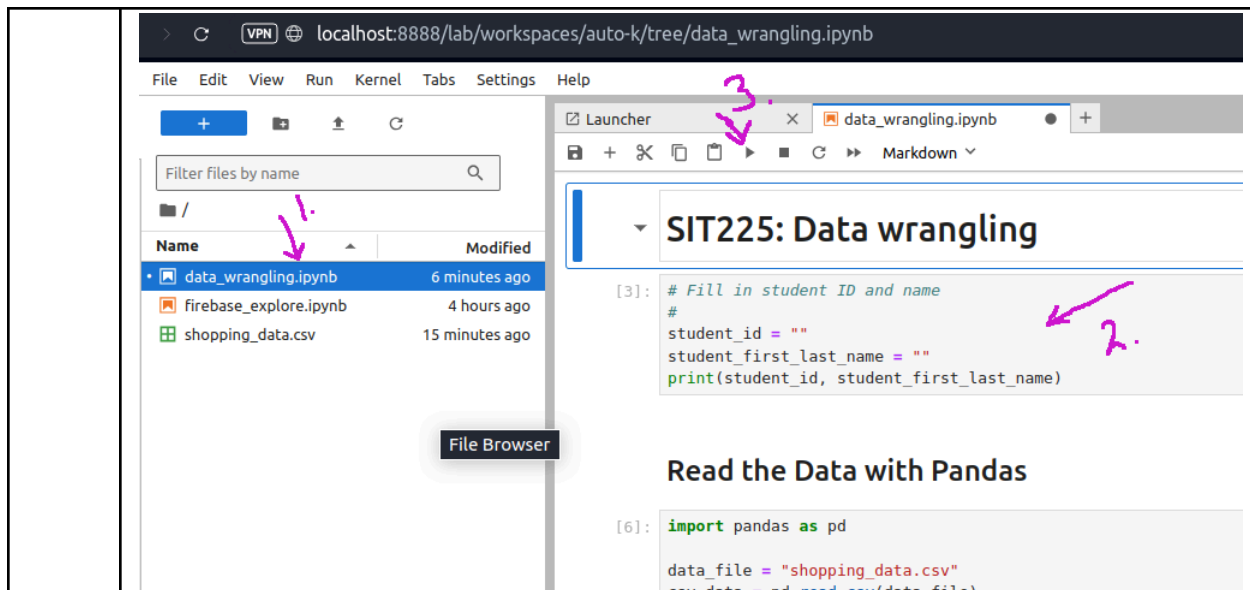
No hardware is required.

Software Required

Python 3
Pandas Python library

Steps

Step	Action
1	Install Pandas using the command below. Most likely you already have Pandas installed if you have installed Python using Anaconda disribution (https://www.anaconda.com/download). <pre>\$ pip install pandas</pre> A Python notebook is shared in the GitHub link (https://github.com/deakin-deep-dreamer/sit225/tree/main/week_5). There will be a data_wrangling.ipynb, shopping_data.csv and shopping_data_missingvalue.csv files among others. Download the week_5 folder in your computer, open a command prompt in that folder, and write the command below in the command line: <pre>\$ jupyter lab</pre> This will open Python Jupyter Notebook where in the left panel you can see the files (labeled as 1 in figure).



Each cell contains Python code (labeled as 2 in figure), you can run a cell by clicking on the cell, so the cursor appears in that cell and then click on the play button at the top of the panel (labeled as 3 in the figure).

2 **Question:** Run each cell to produce output. Follow instructions in the notebook to complete codes in some of the cells. Convert the notebook to PDF from menu File > Save and Export Notebook As > PDF. Convert this activity sheet to PDF and merge with the notebook PDF.

Answer: There is no answer to write here. You have to answer in the Jupyter Notebook.

3 **Question:** Once you went through the cells in the Notebook, you now have a basic understanding of data wrangling. Pandas are a powerful tool and can be used for reading CSV data. Can you use Pandas in reading sensor CSV data that you generated earlier? Describe if any modification you think necessary?

Answer: Yes, Pandas can read the earlier generated CSV sensor data. No modification would be needed to simply *read* that file.

4 **Question:** What do you understand of the Notebook section called Handling Missing Value? Discuss in group and briefly summarise different missing value imputation methods and their applicability on different data conditions.

Answer: I understand that we can generally either delete the column, or infer the missing value in various ways. The way in which we infer data should depend on the characteristics of the data.

- | | |
|--|---|
| | <ul style="list-style-type: none">- Column Deletion: Should only be done if the column has too many missing values and is not critical to data analysis- Imputation (inference):<ul style="list-style-type: none">- Trending data: Interpolate (infer) along the trend- Seasonal data (repeating patterns): Use equivalent points in previous/next cycles- Both: Impute based on trend and repeating patterns. E.g, 1, 2, 3, 2, 3, 4, 3, 4, 5, 4, N/A, 6 - we see seasonality but also an upward trend. 5 would be the best fit as we look at the next value in the cycle, as well as impute based on the upward trend that is evident over all cycles. (linear interpolation) |
|--|---|